ChatGPT

Explain Java Interviews

You



1000159052.jpg
Image

Explain all interviews level

ChatGPT

Java Interview Preparation – Complete Explanation of All 22 Questions

I will explain all 22 questions from your screenshot at a **5-year experienced Java Developer interview level**.

For every question, I will cover:

1. Basic definition.
2. Internal working and concepts.
3. Java code examples.
4. Real-world project usage.
5. Important interview follow-up questions and tricky points.

These explanations will help you answer both theoretical and practical questions in a Java, Spring Boot, and microservices interview.

Take your curiosity further
Scan to get the ChatGPT app

1. Core Java & OOP Concepts

1. What is a Constructor in Java?

A constructor is a special block of code used to initialize an object when it is created.

Whenever we create an object using the `new` keyword, Java invokes its constructor.

Important characteristics

1. Constructor name must be the same as the class name.
2. It does not have a return type, not even `void` .
3. It executes automatically during object creation.
4. Constructors can be overloaded.
5. Constructors cannot be overridden.
6. Constructors are not inherited.
7. Constructors can have access modifiers such as public, private, protected, and default.

Types of constructors

1. Default constructor

If you don't declare any constructor, the Java compiler provides a default no-argument constructor.

```
class Employee {

    int id;
    String name;

}
```

Java conceptually provides:

Take your curiosity further
Scan to get the ChatGPT app

```
Employee() {
    super();
}
```

Important: If you declare any constructor yourself, the compiler does not automatically provide a default constructor.

2. No-argument constructor

```
class Employee {

    int id;
    String name;

    Employee() {
        id = 101;
        name = "Rahul";
    }
}
```

3. Parameterized constructor

```
class Employee {

    int id;
    String name;

    Employee(int id, String name) {
        this.id = id;
        this.name = name;
    }

    public static void main(String[] args) {
```

Take your curiosity further
Scan to get the ChatGPT app

```
        Employee emp = new Employee(101, "Rahul");

        System.out.println(emp.id);
        System.out.println(emp.name);
    }
}
```

What is constructor overloading?

Creating multiple constructors with different parameter lists.

```
class Employee {

    int id;
    String name;

    Employee() {
        System.out.println("Default constructor");
    }

    Employee(int id) {
        this.id = id;
    }

    Employee(int id, String name) {
        this.id = id;
        this.name = name;
    }
}
```

Real-world Spring Boot example

In Spring Boot, constructor injection is commonly used for dependency injection.

Take your curiosity further
Scan to get the ChatGPT app

```
@Service
class EmployeeService {

    private final EmployeeRepository repository;

    public EmployeeService(EmployeeRepository repository) {
        this.repository = repository;
    }

}
```

Spring provides the required repository object through the constructor.

Interview answer: A constructor initializes an object and establishes its initial state. In enterprise applications, constructor injection is preferred because it makes dependencies explicit and supports immutable fields and easier unit testing.

2. Difference between final, finally, and finalize()

This is a very common Java interview question.

Although their names are similar, they have completely different purposes.

Feature	final	finally	finalize()
Type	Keyword	Block	Method
Purpose	Restricts modification	Executes cleanup code	Legacy GC-related callback
Used with	Variable, method, class	try-catch	Object
Inheritance	Restricts overriding/inheritance	Not applicable	Can be overridden

Take your curiosity further
Scan to get the ChatGPT app

Current recommendation	Commonly used	Used where appropriate	Avoid

A. final keyword

Final variable

Once assigned, its value cannot be reassigned.

```
final int age = 25;

// age = 30; // Compilation error
```

Final method

A child class cannot override a final method.

```
class Parent {

    final void display() {
        System.out.println("Parent method");
    }
}

class Child extends Parent {

    // Cannot override display()
}
```

Final class

A final class cannot be extended.

Take your curiosity further
Scan to get the ChatGPT app

```
final class Vehicle {

}

// class Car extends Vehicle {} // Error
```

Example: Java's `String` class is final.

B. finally block

The finally block is used with exception handling.

It normally executes whether an exception occurs or not.

```
public class Test {

    public static void main(String[] args) {

        try {

            int result = 10 / 0;

        } catch (Exception e) {

            System.out.println("Exception occurred");

        } finally {

            System.out.println("Finally executed");
        }
    }
}
```

Output:

Take your curiosity further
Scan to get the ChatGPT app

```
Exception occurred
Finally executed
```

Real-world usage:

- Closing database connections.
- Releasing resources.
- Cleanup operations.

However, prefer try-with-resources for `AutoCloseable` resources.

C. finalize() method

`finalize()` was historically called by the garbage collector before reclaiming an object, but its execution was never guaranteed.

```
@Override
protected void finalize() throws Throwable {

    System.out.println("Object cleanup");

}
```

Important: Finalization is deprecated for removal in modern Java. Do not use it for resource management.

Use:

- try-with-resources
- AutoCloseable
- Explicit resource cleanup

Interview trick: Can finally execute without catch?

Yes.

Take your curiosity further
Scan to get the ChatGPT app

```
try {

    System.out.println("Try");

} finally {

    System.out.println("Finally");
}
```

Can finally fail to execute? Yes, for example, when the JVM terminates abruptly using `System.exit()`.

3. Method Overloading vs Method Overriding

These are two important concepts of polymorphism.

A. Method overloading

When multiple methods have the same name but different parameter lists within a class, it is called method overloading.

It is compile-time polymorphism.

```
class Calculator {

    int add(int a, int b) {
        return a + b;
    }

    int add(int a, int b, int c) {
        return a + b + c;
    }
```

Take your curiosity further
Scan to get the ChatGPT app

```
    double add(double a, double b) {
        return a + b;
    }
}
```

The compiler determines which method to execute based on the arguments.

B. Method overriding

When a child class provides its own implementation of a method inherited from the parent class, it is called method overriding.

It is runtime polymorphism.

```
class Payment {

    void pay() {
        System.out.println("General payment");
    }
}

class UpiPayment extends Payment {

    @Override
    void pay() {
        System.out.println("UPI payment");
    }
}

class Main {

    public static void main(String[] args) {

        Payment payment = new UpiPayment();

        payment.pay();
```

Take your curiosity further
Scan to get the ChatGPT app

```
    }
}
```

Output:

```
UPI payment
```

Internal working

```
Payment payment = new UpiPayment();
```

Here:

- Reference type = Payment.
- Actual object type = UpiPayment.
- The overridden method is selected at runtime based on the actual object.

This is called dynamic method dispatch.

Difference

Overloading	Overriding
Compile-time polymorphism	Runtime polymorphism
Same method name, different parameters	Same method signature (with compatible return type)
Usually within the same class	Requires inheritance
Return type alone cannot differentiate methods	Covariant return types are allowed
Static methods can be overloaded	Static methods are hidden, not overridden

Real-world example: In a banking application, a common `Payment` interface can have implementations for UPI, card, and net banking.

Take your curiosity further
Scan to get the ChatGPT app

4. Can static methods be overridden?

No. Static methods cannot be overridden in Java.

They belong to the class rather than an individual object.

If a child class declares a static method with the same signature, it is called method hiding.

Example

```
class Parent {

    static void display() {
        System.out.println("Parent static method");
    }
}

class Child extends Parent {

    static void display() {
        System.out.println("Child static method");
    }
}

class Main {

    public static void main(String[] args) {

        Parent obj = new Child();

        obj.display();
    }
}
```

Take your curiosity further
Scan to get the ChatGPT app

Output:

```
Parent static method
```

Why?

Because static method resolution happens using the reference type at compile time, not the actual object type.

Compare with instance methods:

```
class Parent {

    void show() {
        System.out.println("Parent");
    }
}

class Child extends Parent {

    @Override
    void show() {
        System.out.println("Child");
    }
}
```

Now:

```
Parent obj = new Child();
obj.show();
```

Output:

Take your curiosity further
Scan to get the ChatGPT app

```
Child
```

Interview point: Static methods support overloading but not overriding.

5. Can a constructor be declared final?

No, a constructor cannot be declared final.

Why?

The purpose of final is to prevent overriding.

But constructors cannot be overridden in Java anyway.

Therefore, declaring a constructor final is meaningless and not allowed.

```
class Employee {

    // final Employee() {} // Compilation error

}
```

Follow-up: Can a constructor be private?

Yes.

```
class Singleton {

    private static final Singleton INSTANCE =
            new Singleton();

    private Singleton() {
    }
```

Take your curiosity further
Scan to get the ChatGPT app

```
    public static Singleton getInstance() {
        return INSTANCE;
    }
}
```

Private constructors are commonly used in:

- Singleton design patterns.
- Utility classes.
- Factory-controlled object creation.

6. Can we overload the main() method?

Yes. We can overload the main method.

However, the JVM looks for the standard entry point:

```
public static void main(String[] args)
```

Example

```
class Main {

    public static void main(String[] args) {

        System.out.println("Original main");

        main(10);
    }

    public static void main(int number) {
```

Take your curiosity further
Scan to get the ChatGPT app

```
        System.out.println("Overloaded main");
    }
}
```

Output:

```
Original main
Overloaded main
```

Another example:

```
public static void main(String args) {
    System.out.println("String main");
}
```

This is a valid overloaded method, but it is not the standard JVM entry point.

Interview answer: Java supports main method overloading, but JVM starts execution from the standard main method signature.

7. Can we create an object of an interface?

Directly, no.

An interface cannot be instantiated using the `new` keyword.

However, we can create an object of a class that implements the interface.

Example 1: Implementation class

```
interface Payment {
```

Take your curiosity further
Scan to get the ChatGPT app

```
    void pay();
}

class UpiPayment implements Payment {

    @Override
    public void pay() {

        System.out.println("Payment through UPI");
    }
}

class Main {

    public static void main(String[] args) {

        Payment payment = new UpiPayment();

        payment.pay();
    }
}
```

Here:

```
Payment payment = new UpiPayment();
```

The reference is of interface type, but the object is of implementation class type.

Example 2: Anonymous inner class

```
interface Vehicle {

    void start();
}
```



Take your curiosity further

Scan to get the ChatGPT app



```
class Main {

    public static void main(String[] args) {

        Vehicle vehicle = new Vehicle() {

            @Override
            public void start() {
                System.out.println("Vehicle started");
            }
        };

        vehicle.start();
    }
}
```

Example 3: Lambda expression

For a functional interface:

```
Runnable task = () ->
        System.out.println("Task running");

task.run();
```

Real-world usage: Spring Boot uses interfaces extensively for loose coupling.

For example:

```
public interface EmployeeService {
    void saveEmployee();
}
```

Take your curiosity further
Scan to get the ChatGPT app

You can have:

```
@Service
class EmployeeServiceImpl implements EmployeeService {
    // Implementation
}
```

8. Can a final variable be reassigned?

No. Once a final variable has been initialized, it cannot be reassigned.

But there is an important difference between primitive variables and object references.

Primitive example

```
final int number = 10;

// number = 20; // Error
```

Object reference example

```
final StringBuilder sb = new StringBuilder("Hello");

sb.append(" Java");

System.out.println(sb);
```

Output:

```
Hello Java
```

Take your curiosity further
Scan to get the ChatGPT app

Why is this allowed?

Because the reference cannot point to another object, but the object's internal state may still change.

This is called reference immutability, not object immutability.

Another example

```
final StringBuilder sb = new StringBuilder("Java");

// sb = new StringBuilder("Python"); // Error

sb.append(" Developer"); // Allowed
```

Interview follow-up: How to make an object immutable?

1. Declare the class final.
2. Make fields private and final.
3. Initialize fields through the constructor.
4. Do not provide setters.
5. Use defensive copies for mutable objects.

2. Java 8 & Functional Programming

9. What are the important features introduced in Java 8?

Java 8 was a major release because it introduced functional programming capabilities into Java.

Important features:

Feature	Purpose

Take your curiosity further
Scan to get the ChatGPT app

Lambda expressions	Write concise functional code
Functional interfaces	Support lambda expressions
Stream API	Process collections declaratively
Method references	Reuse existing methods
Default methods	Add methods to interfaces with implementations
Static interface methods	Define utility methods in interfaces
Optional	Represent potentially absent values
CompletableFuture	Asynchronous programming
New Date and Time API	Better date/time handling
forEach()	Iterate using functional style

Example: Before Java 8

```
List<Integer> numbers = Arrays.asList(1, 2, 3, 4, 5);

for (Integer number : numbers) {

    if (number % 2 == 0) {
        System.out.println(number);
    }
}
```

After Java 8

```
numbers.stream()
       .filter(n -> n % 2 == 0)
       .forEach(System.out::println);
```

Take your curiosity further
Scan to get the ChatGPT app

Output:

```
2
4
```

Why did Java introduce Stream API?

To make collection processing:

- More readable.
- More declarative.
- Easier to compose.
- Suitable for parallel processing.

Real-world project example

Suppose we have an employee list and need employees whose salary is greater than 50,000.

```
List<Employee> result = employees.stream()
        .filter(e -> e.getSalary() > 50000)
        .collect(Collectors.toList());
```

This is commonly used in Spring Boot service-layer business logic.

10. What is CompletableFuture and what are its features?

`CompletableFuture` is a Java 8 class used for asynchronous and non-blocking task composition.

It implements the `Future` and `CompletionStage` interfaces.

It allows us to execute tasks asynchronously without waiting for every task sequentially.

Take your curiosity further
Scan to get the ChatGPT app

Why do we need it?

Imagine a banking application that needs to fetch:

1. Customer details.
2. Account details.
3. Transaction history.

Suppose all three operations take two seconds.

Sequential execution:

Sequential execution

2 sec Customer details

2 sec Account details

2 sec Transaction history

Total: approximately 6 seconds

Parallel execution:

Parallel execution

2 sec Customer details

2 sec Account details

2 sec Transaction history

Total: approximately 2 seconds, assuming sufficient execution resources and similar task durations.

Take your curiosity further
Scan to get the ChatGPT app

Basic example

```
import java.util.concurrent.CompletableFuture;

public class Main {

    public static void main(String[] args) {

        CompletableFuture<String> future =
                CompletableFuture.supplyAsync(() -> {

                    return "Employee data fetched";

                });

        future.thenAccept(result -> {
            System.out.println(result);
        });

        future.join();
    }
}
```

Important methods

1. runAsync()

Executes an asynchronous task that does not return a result.

```
CompletableFuture<Void> future =
    CompletableFuture.runAsync(() -> {
        System.out.println("Running asynchronously");
    });
```

2. supplyAsync()

Take your curiosity further
Scan to get the ChatGPT app

Executes an asynchronous task and returns a result.

```
CompletableFuture<String> future =
    CompletableFuture.supplyAsync(() -> "Hello Java");
```

3. thenApply()

Transforms the result.

```
CompletableFuture<Integer> future =
    CompletableFuture.supplyAsync(() -> 10)
        .thenApply(n -> n * 2);
```

Result: 20.

4. thenAccept()

Consumes the result without returning another result.

```
future.thenAccept(System.out::println);
```

5. thenRun()

Executes another action without consuming the previous result.

```
future.thenRun(() ->
    System.out.println("Completed")
);
```

6. thenCombine()

Combines results from two independent futures.

Take your curiosity further
Scan to get the ChatGPT app

```
CompletableFuture<Integer> f1 =
    CompletableFuture.supplyAsync(() -> 10);

CompletableFuture<Integer> f2 =
    CompletableFuture.supplyAsync(() -> 20);

CompletableFuture<Integer> result =
    f1.thenCombine(f2, Integer::sum);

System.out.println(result.join());
```

Output:

```
30
```

7. exceptionally()

Handles exceptions.

```
CompletableFuture<Integer> future =
    CompletableFuture.supplyAsync(() -> {
        if (true) {
            throw new RuntimeException("Error");
        }
        return 100;
    }).exceptionally(ex -> 0);
```

8. allOf()

Waits for all futures to complete.

9. anyOf()

Completes when any one of the supplied futures completes.

Take your curiosity further
Scan to get the ChatGPT app

10. join() vs get()

join()	get()
Does not declare checked exceptions	Throws checked exceptions
Throws CompletionException for exceptional completion	Throws ExecutionException
Used frequently with CompletableFuture	Inherited from Future

Real-world Spring Boot example

Suppose your dashboard calls multiple microservices.

```
@Service
public class DashboardService {

    private final Executor executor;

    public DashboardService(
            @Qualifier("applicationExecutor")
            Executor executor) {
        this.executor = executor;
    }

    public CompletableFuture<String> getCustomer() {

        return CompletableFuture.supplyAsync(
                () -> "Customer details",
                executor
        );
    }

    public CompletableFuture<String> getAccount() {

        return CompletableFuture.supplyAsync(
                () -> "Account details",
```

Take your curiosity further
Scan to get the ChatGPT app

```
                executor
        );
    }
}
```

You can combine the results:

```
public CompletableFuture<String> getDashboard() {

    return getCustomer().thenCombine(
            getAccount(),
            (customer, account) ->
                    customer + " | " + account
    );
}
```

Senior-level interview point: `CompletableFuture` does not automatically make an application non-blocking end-to-end. Blocking database or HTTP calls can still consume worker threads. Use an appropriately configured executor, timeouts, and exception handling.

11. What is a Lambda Expression?

A lambda expression is an anonymous function that allows us to implement a functional interface in a concise way.

It was introduced in Java 8.

Syntax

```
(parameters) -> expression
```

Or:

Take your curiosity further
Scan to get the ChatGPT app

```
(parameters) -> {
    // Multiple statements
}
```

Before Java 8

```
Runnable runnable = new Runnable() {

    @Override
    public void run() {
        System.out.println("Running");
    }
};
```

After Java 8

```
Runnable runnable = () -> {
    System.out.println("Running");
};
```

Different lambda examples

No parameter:

```
() -> System.out.println("Hello");
```

One parameter:

```
x -> x * x;
```

Take your curiosity further
Scan to get the ChatGPT app

Two parameters:

```
(a, b) -> a + b;
```

Multiple statements:

```
(a, b) -> {

    int result = a + b;

    return result;
};
```

Real-world example

```
List<String> names =
    Arrays.asList("Rahul", "Amit", "Shiva");

names.forEach(name ->
    System.out.println(name)
);
```

Important interview point

A lambda expression requires a functional interface as its target type.

It cannot directly implement an interface having multiple unrelated abstract methods.

12. What is a Functional Interface?

A functional interface is an interface that contains exactly one abstract method.

Take your curiosity further
Scan to get the ChatGPT app

It is also known as a Single Abstract Method (SAM) interface.

It can have:

- One abstract method.
- Multiple default methods.
- Multiple static methods.
- Methods inherited from `Object` do not count as additional abstract methods.

Example

```
@FunctionalInterface
interface Calculator {

    int add(int a, int b);
}
```

Implementation using lambda:

```
class Main {

    public static void main(String[] args) {

        Calculator calculator =
                (a, b) -> a + b;

        System.out.println(
                calculator.add(10, 20)
        );
    }
}
```

Output:

Take your curiosity further
Scan to get the ChatGPT app

```
30
```

Why use @FunctionalInterface?

It tells the compiler that the interface is intended to be functional.

If you add another unrelated abstract method, compilation fails.

```
@FunctionalInterface
interface Test {

    void show();

    // void display(); // Compilation error
}
```

Real-world usage

Functional interfaces are heavily used in:

- Streams.
- Lambda expressions.
- Event handling.
- Asynchronous processing.
- Spring application callbacks.

13. What are the built-in functional interfaces in Java?

Java provides several functional interfaces inside:

```
java.util.function
```

Take your curiosity further
Scan to get the ChatGPT app

The most important ones are:

1. Predicate

Accepts one argument and returns a boolean.

```
Predicate<Integer> predicate =
        n -> n > 10;

System.out.println(predicate.test(20));
```

Output: `true`

Real-world usage: Filtering employees based on salary.

2. Function

Accepts one argument and returns another value.

```
Function<String, Integer> function =
        name -> name.length();

System.out.println(function.apply("Rahul"));
```

Output: `5`

Real-world usage: Converting entity objects into DTOs.

3. Consumer

Accepts one argument and returns nothing.

```
Consumer<String> consumer =
        name -> System.out.println(name);
```

Take your curiosity further
Scan to get the ChatGPT app

```
consumer.accept("Java");
```

Real-world usage: Logging or processing an object.

4. Supplier

Does not accept arguments but returns a value.

```
Supplier<Double> supplier =
        () -> Math.random();

System.out.println(supplier.get());
```

Real-world usage: Generating values or supplying objects.

5. BiPredicate

Accepts two arguments and returns boolean.

```
BiPredicate<Integer, Integer> check =
        (a, b) -> a > b;

System.out.println(check.test(20, 10));
```

6. BiFunction

Accepts two arguments and returns a result.

```
BiFunction<Integer, Integer, Integer> sum =
        (a, b) -> a + b;
```

Take your curiosity further
Scan to get the ChatGPT app

7. BiConsumer

Accepts two arguments and returns nothing.

```
BiConsumer<String, Integer> employee =
        (name, age) ->
                System.out.println(name + " " + age);
```

Quick interview revision

Interface	Input	Output	Main method
Predicate	1	Boolean	test()
Function	1	Result	apply()
Consumer	1	Void	accept()
Supplier	0	Result	get()
BiPredicate	2	Boolean	test()
BiFunction	2	Result	apply()
BiConsumer	2	Void	accept()
UnaryOperator	1	Same type	apply()
BinaryOperator	2	Same type	apply()

Interview trick: Predicate is used for conditions, Function for transformations, Consumer for actions, and Supplier for providing values.

Take your curiosity further
Scan to get the ChatGPT app

3. Java Collections Framework

14. What is a HashMap?

`HashMap` is a class in Java that implements the `Map` interface.

It stores data in key-value pairs.

Example:

```
Map<Integer, String> employees =
        new HashMap<>();

employees.put(101, "Rahul");
employees.put(102, "Amit");
employees.put(103, "Shiva");

System.out.println(employees.get(101));
```

Output:

```
Rahul
```

Important characteristics

1. Stores data in key-value pairs.
2. Keys must be unique.
3. Values can be duplicated.
4. Allows one null key.
5. Allows multiple null values.
6. Does not guarantee insertion order.
7. Not thread-safe by default.
8. Average lookup and insertion complexity is O(1), assuming good hash distribution.

Take your curiosity further
Scan to get the ChatGPT app

Important HashMap methods

```
put()
get()
remove()
containsKey()
containsValue()
putIfAbsent()
getOrDefault()
keySet()
values()
entrySet()
```

Example: Iterate HashMap

```
Map<Integer, String> map = new HashMap<>();

map.put(1, "Java");
map.put(2, "Spring Boot");

for (Map.Entry<Integer, String> entry :
        map.entrySet()) {

    System.out.println(
            entry.getKey() + " " +
            entry.getValue()
    );
}
```

HashMap vs Hashtable vs ConcurrentHashMap

Feature	HashMap	Hashtable	ConcurrentHashMap
Thread-safe	No	Yes	Yes

Take your curiosity further
Scan to get the ChatGPT app

Null key	One	No	No
Null values	Multiple	No	No
Concurrency	No built-in protection	Coarse-grained synchronization	Concurrent operations
Performance	Good for single-threaded use	Often lower under contention	Designed for concurrent access

Real-world usage:

- Caching.
- Counting frequencies.
- Storing employee details.
- Configuration lookup.
- In-memory mappings.

15. Explain the internal working of HashMap.

Very important for 5-year experience interviews

This is one of the most frequently asked Java Collections questions.

You should be able to explain the complete internal flow of `put()` and `get()`.

Internal architecture

HashMap internally uses an array of buckets.

Each bucket can contain:

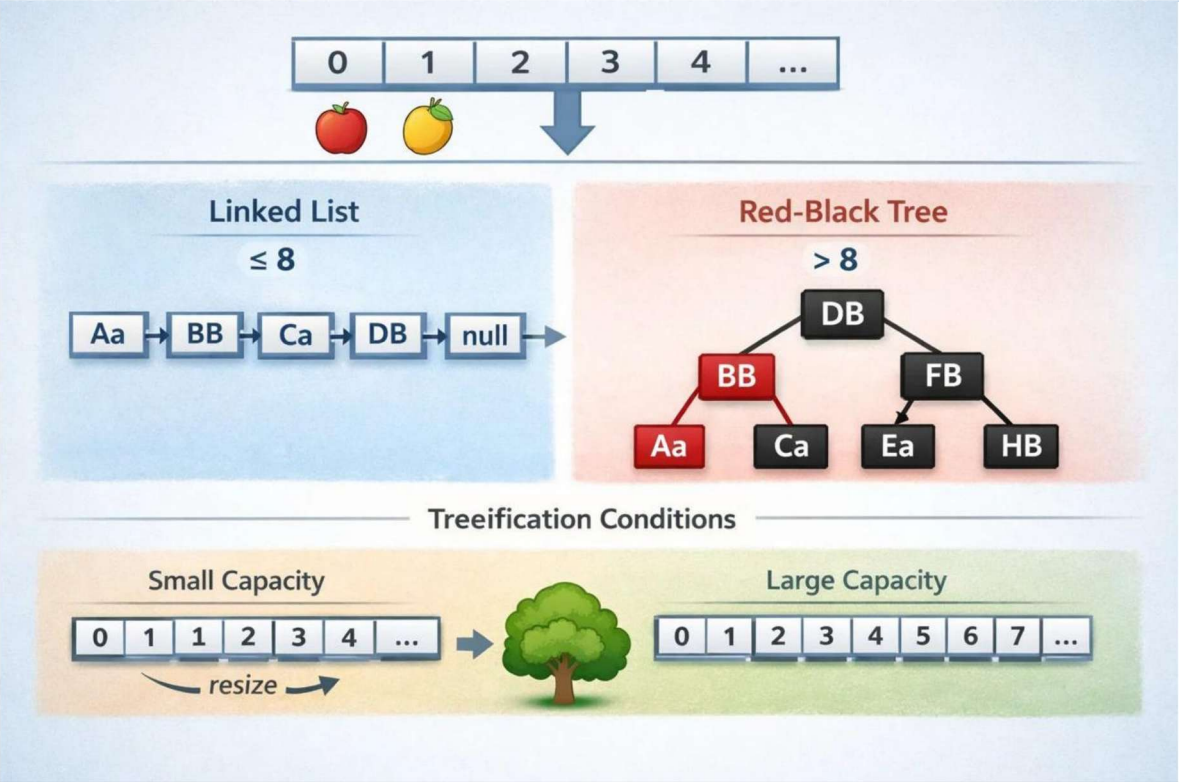
- No node.
- One node.
- A linked list of nodes.
- A balanced red-black tree when collision handling crosses the relevant thresholds.



Take your curiosity further

Scan to get the ChatGPT app





Conceptually, each node stores:

```
static class Node<K,V> {

    final int hash;

    final K key;

    V value;

    Node<K,V> next;
}
```

A. Internal working of put()

Suppose we execute:

Take your curiosity further
Scan to get the ChatGPT app

```
Map<Integer, String> map = new HashMap<>();

map.put(101, "Rahul");
```

The internal process:

Step 1: Calculate hash

Java calculates the hash of the key.

For standard Java 8+ HashMap, the spread operation is conceptually:

```
static final int hash(Object key) {

    int h;

    return (key == null)
            ? 0
            : (h = key.hashCode()) ^ (h >>> 16);
}
```

This mixes the high bits into the lower bits.

Step 2: Calculate bucket index

```
index = (n - 1) & hash;
```

Here `n` represents the capacity of the internal array.

Step 3: Check bucket

- If bucket is empty, insert the new node.
- If bucket is occupied, check whether the key already exists.
- If the key exists, update its value.

Take your curiosity further
Scan to get the ChatGPT app

- Otherwise, handle the collision.

Step 4: Handle collision

Hash collision means different keys map to the same bucket.

HashMap uses:

- Linked list for ordinary collisions.
- Red-black tree for sufficiently large collision bins, when capacity conditions are met.

In Java 8+, treeification normally occurs at a bin size of 8 when table capacity is at least 64. Otherwise, HashMap may resize instead.

B. Internal working of get()

Suppose:

```
String name = map.get(101);
```

Steps:

1. Calculate the key hash.
2. Calculate bucket index.
3. Find the corresponding bucket.
4. Compare the stored key's hash.
5. Check equality using `equals()`.
6. Return the value if a matching key is found.

C. Why are hashCode() and equals() important?

They are fundamental to correct HashMap key behavior.

The contract:

- If two objects are equal according to `equals()`, they must have the same hash code.

Take your curiosity further
Scan to get the ChatGPT app

- Two objects with the same hash code do not necessarily have to be equal.

Example:

```
class Employee {

    private int id;

    Employee(int id) {
        this.id = id;
    }

    @Override
    public int hashCode() {
        return Integer.hashCode(id);
    }

    @Override
    public boolean equals(Object obj) {

        if (this == obj) return true;

        if (!(obj instanceof Employee)) return false;

        Employee other = (Employee) obj;

        return this.id == other.id;
    }
}
```

Now:

```
Map<Employee, String> map = new HashMap<>();

map.put(new Employee(101), "Rahul");
```



Take your curiosity further

Scan to get the ChatGPT app



```
System.out.println(
        map.get(new Employee(101))
);
```

Output:

```
Rahul
```

Because both Employee objects are considered equal.

D. What is the load factor?

The default initial capacity is 16, and the default load factor is 0.75.

Resize threshold:

$$16 \times 0.75 = 12$$

When the number of entries exceeds the threshold, HashMap generally resizes its internal table.

Capacity usually doubles.

For example:

```
Initial capacity = 16
Threshold = 12

After resize:
Capacity = 32
Threshold = 24
```

Take your curiosity further
Scan to get the ChatGPT app

E. What happens if two keys have the same hashCode()?

HashMap handles collisions.

It checks equality to identify whether the key is already present.

If keys are different, both entries can exist in the same bucket.

F. What happens if a key is mutable?

Suppose you insert an object as a key and later modify a field used in its `hashCode()` or `equals()` implementation.

A subsequent lookup may fail because the key now maps to a different bucket.

Best practice: Use immutable objects as HashMap keys.

G. Time complexity

Operation	Average case	Worst case
put()	O(1)	O(n), generally
get()	O(1)	O(n), generally
remove()	O(1)	O(n), generally

For well-distributed keys and treeified bins, tree operations can be O(log n), subject to the key comparison and implementation details.

Interview-ready answer:

"HashMap internally uses an array of buckets. It calculates the hash code of the key, spreads the hash, and determines the bucket index using the bitwise AND operation. If the bucket is empty, it inserts a node. If a collision occurs, it checks hashCode and equals and stores the entry in a linked list or a red-black tree under the applicable conditions. It also resizes when the threshold determined by capacity and load factor is exceeded."



Take your curiosity further

Scan to get the ChatGPT app



4. Multithreading & Concurrency

16. What is Multithreading in Java?

Multithreading is the ability of a program to execute multiple threads concurrently.

A thread is a lightweight unit of execution within a process.

Example

Imagine an e-commerce application.

At the same time, it may:

- Process a customer payment.
- Send an email.
- Update inventory.
- Generate an invoice.

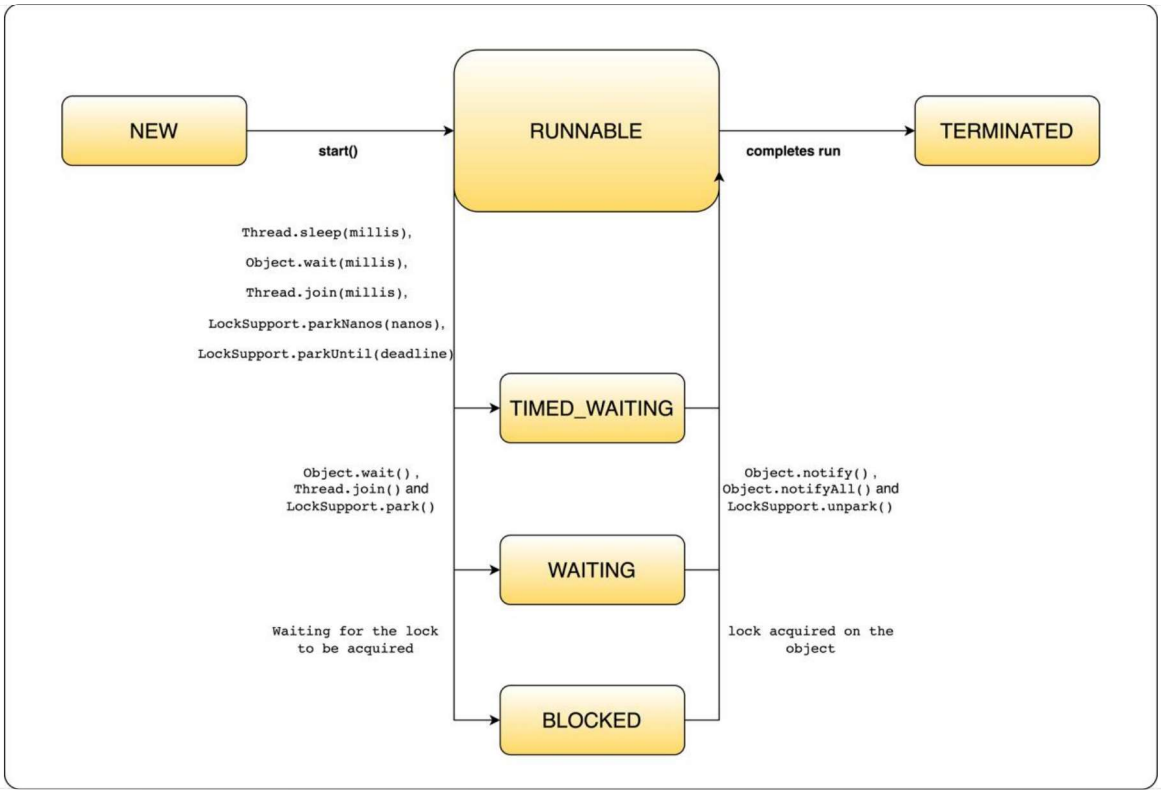
Instead of executing all these operations sequentially, independent tasks can run concurrently.

Process vs Thread

Process	Thread
Independent running program	Execution unit inside a process
Has its own address space	Shares process memory
More expensive to create	Generally lighter
Communicates through IPC mechanisms	Can communicate through shared memory

Take your curiosity further
Scan to get the ChatGPT app

Java thread lifecycle



Java defines these thread states:

1. NEW – Thread object created.
2. RUNNABLE – Ready or executing.
3. BLOCKED – Waiting to acquire a monitor lock.
4. WAITING – Waiting indefinitely for another thread's action.
5. TIMED_WAITING – Waiting for a specified time.
6. TERMINATED – Execution completed.

Example

```
class MyThread extends Thread {

    @Override
    public void run() {
```

Take your curiosity further
Scan to get the ChatGPT app

```
        System.out.println(
                Thread.currentThread().getName()
        );
    }
}

class Main {

    public static void main(String[] args) {

        MyThread t1 = new MyThread();

        t1.start();
    }
}
```

start() vs run()

Very important interview question.

```
t1.start();
```

Creates a new execution path and eventually invokes `run()` on that thread.

```
t1.run();
```

Calls the method like an ordinary Java method in the current thread. It does not create a new thread.

What is synchronization?

Synchronization controls access to shared resources to prevent race conditions.

Take your curiosity further
Scan to get the ChatGPT app

```
class Counter {

    private int count = 0;

    public synchronized void increment() {

        count++;
    }

    public int getCount() {
        return count;
    }
}
```

Here, only one thread at a time can execute the synchronized instance method for the same object monitor.

Important concurrency problems

- Race condition.
- Deadlock.
- Starvation.
- Thread contention.
- Visibility problems.

17. How can multithreading be achieved in Java?

Multithreading can be achieved using several Java concurrency mechanisms.

The main approaches are:

1. Thread class.
2. Runnable interface.
3. Callable interface with ExecutorService.

Take your curiosity further
Scan to get the ChatGPT app

4. Thread pools.
5. CompletableFuture.
6. ScheduledExecutorService.

Example 1: Thread class

```
class MyThread extends Thread {

    public void run() {

        System.out.println("Thread running");
    }
}

class Main {

    public static void main(String[] args) {

        MyThread t1 = new MyThread();

        t1.start();
    }
}
```

Example 2: Runnable interface

```
class MyTask implements Runnable {

    @Override
    public void run() {

        System.out.println("Runnable task running");
    }
}
```

Take your curiosity further
Scan to get the ChatGPT app

```
class Main {

    public static void main(String[] args) {

        Thread t1 = new Thread(new MyTask());

        t1.start();
    }
}
```

Example 3: ExecutorService

This is widely used in real-world enterprise applications.

```
import java.util.concurrent.*;

public class Main {

    public static void main(String[] args) {

        ExecutorService executor =
                Executors.newFixedThreadPool(3);

        executor.submit(() -> {

            System.out.println(
                    Thread.currentThread().getName()
            );

        });

        executor.shutdown();
    }
}
```

Take your curiosity further
Scan to get the ChatGPT app

Why prefer ExecutorService?

Because it:

- Reuses existing threads.
- Reduces thread creation overhead.
- Manages task execution.
- Provides Future results.
- Supports controlled concurrency.

Real-world example

Suppose a banking application receives 100 requests.

Creating 100 new threads for every request can be expensive.

Instead, a thread pool can manage a limited number of worker threads and queue pending tasks.

Senior interview point: Choose the pool size based on workload, CPU cores, I/O wait, downstream limits, and latency requirements. Avoid unbounded task queues and unbounded thread creation.

18. What are the different ways to create a thread?

There are several commonly discussed ways.

1. Extend Thread class

```
class MyThread extends Thread {

    @Override
    public void run() {
        System.out.println("Thread 1");
    }
}
```

Take your curiosity further
Scan to get the ChatGPT app

Execution:

```
new MyThread().start();
```

2. Implement Runnable

```
class MyTask implements Runnable {

    @Override
    public void run() {
        System.out.println("Thread 2");
    }
}
```

Execution:

```
new Thread(new MyTask()).start();
```

3. Implement Callable

Callable allows returning a value and throwing checked exceptions.

```
import java.util.concurrent.*;

class Main {

    public static void main(String[] args)
            throws Exception {

        ExecutorService executor =
                Executors.newSingleThreadExecutor();
```

Take your curiosity further
Scan to get the ChatGPT app

```
        Callable<Integer> task = () -> 100;

        Future<Integer> future =
                executor.submit(task);

        System.out.println(future.get());

        executor.shutdown();
    }
}
```

Output:

```
100
```

4. Using Lambda with Runnable

```
Thread t = new Thread(() -> {

    System.out.println("Lambda thread");

});

t.start();
```

5. Using ExecutorService

```
ExecutorService executor =
        Executors.newFixedThreadPool(5);

executor.execute(() ->
        System.out.println("Task executed")
);
```

Take your curiosity further
Scan to get the ChatGPT app

```
executor.shutdown();
```

6. Using CompletableFuture

```
CompletableFuture.runAsync(() -> {

    System.out.println("Async task");

});
```

Important comparison

Approach	Return value	Exception handling	Best use
Thread	No direct result	Inside run()	Basic thread creation
Runnable	No direct result	Inside run()	Independent tasks
Callable	Yes	Can throw checked exceptions	Result-producing tasks
ExecutorService	Depends on submitted task	Future	Managed thread pools
CompletableFuture	Yes or Void	Composition and exception stages	Asynchronous workflows

Interview answer: For enterprise applications, I generally prefer ExecutorService or CompletableFuture rather than manually creating threads, because they provide better control over concurrency and resource management.

Take your curiosity further
Scan to get the ChatGPT app

5. JVM, JRE & Memory Management

19. What is JRE? Explain its architecture.

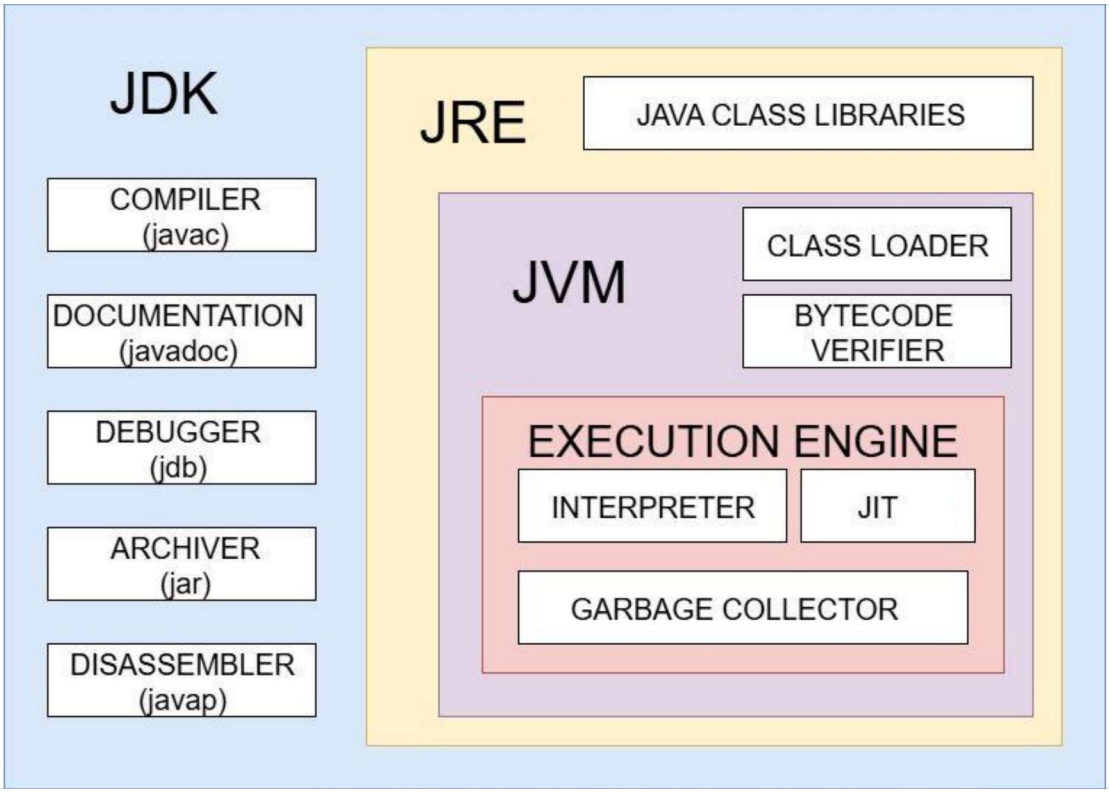
JRE stands for Java Runtime Environment.

It provides the environment required to execute Java applications.

Traditionally, JRE consists of:

1. JVM.
2. Core Java runtime libraries.
3. Supporting runtime components and resources.

Difference between JDK, JRE, and JVM



Component	Meaning	Purpose

Take your curiosity further
Scan to get the ChatGPT app

JDK	Java Development Kit	Develop and run Java applications
JRE	Java Runtime Environment	Run Java applications
JVM	Java Virtual Machine	Execute Java bytecode

A modern JDK distribution generally includes the runtime components needed to execute applications. Since Java 11, Oracle no longer provides a separate traditional JRE download as part of its standard JDK distribution.

How Java executes a program

Step 1: Write Java code.

```
class Hello {

    public static void main(String[] args) {

        System.out.println("Hello Java");
    }
}
```

Step 2: Compile using the Java compiler.

```
javac Hello.java
```

Step 3: Compiler generates bytecode.

```
Hello.class
```

Step 4: JVM loads the class.

Step 5: JVM verifies the bytecode.



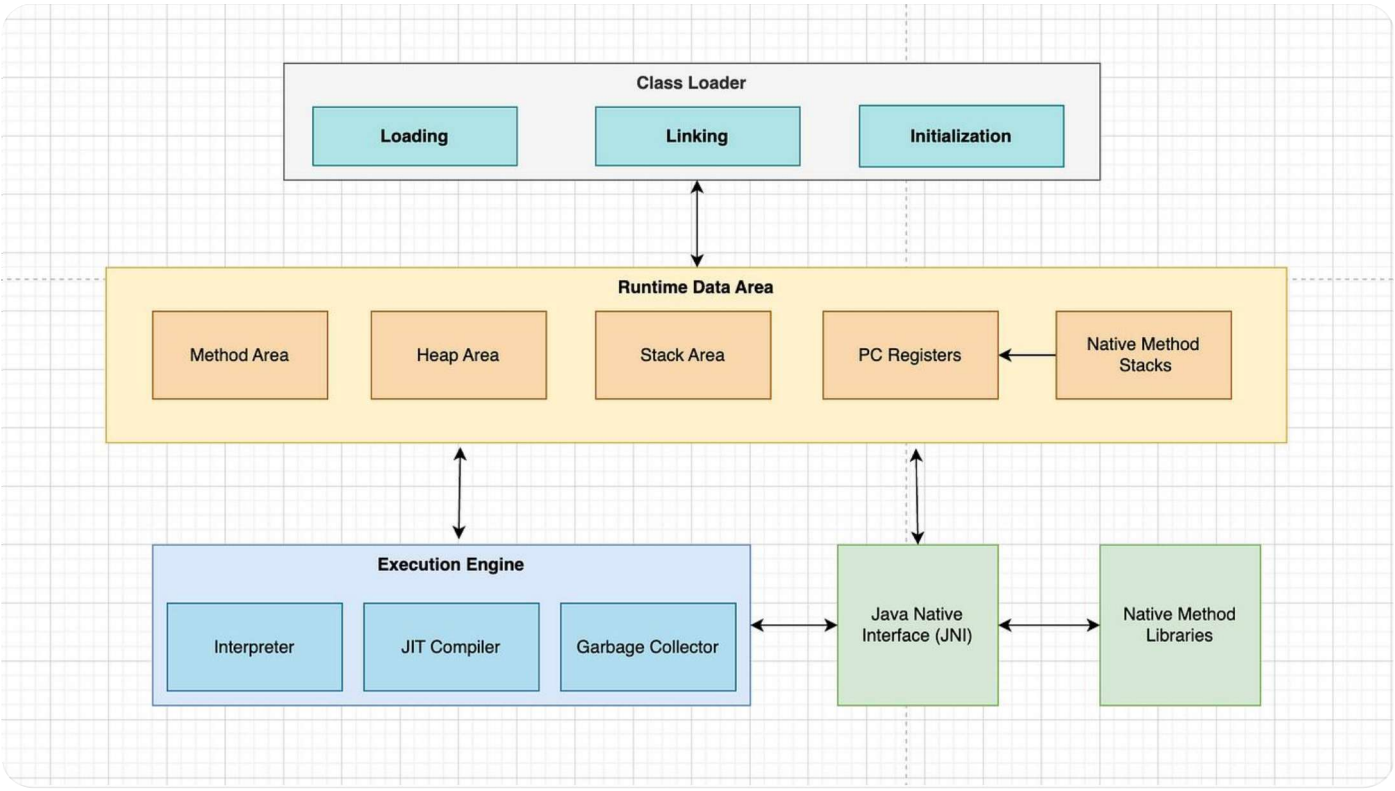
Take your curiosity further

Scan to get the ChatGPT app



Step 6: Execution engine executes the bytecode.

JVM architecture



The main JVM components are:

1. Class Loader Subsystem

Loads Java classes into memory.

It includes:

- Loading.
- Linking.
- Initialization.

2. Runtime Data Areas

Memory is divided into different areas.

Take your curiosity further
Scan to get the ChatGPT app

Memory area	Purpose
Heap	Stores objects and arrays
Java Stack	Stores method frames and local variables
PC Register	Tracks the current instruction for each thread
Method Area	Stores class-level structures and runtime metadata
Native Method Stack	Supports native method execution

3. Execution Engine

Responsible for executing bytecode.

It includes:

- Interpreter.
- JIT compiler.
- Garbage collector.

4. JNI

Java Native Interface allows Java code to interact with native code.

What is JIT compiler?

JIT stands for Just-In-Time compiler.

It identifies frequently executed code and compiles it into native machine code to improve performance.

Interview-ready answer

"JRE provides the runtime environment for executing Java applications. JVM is its core component. JVM loads class files through the class loader, stores runtime data in different memory areas, and executes bytecode through the execution engine. JIT compilation optimizes frequently executed

Take your curiosity further
Scan to get the ChatGPT app

code, while garbage collection manages heap memory."

20. What is a ClassLoader?

A ClassLoader is a JVM component responsible for loading Java classes into memory when required.

For example:

```
Employee emp = new Employee();
```

Before creating the object, JVM needs to load the Employee class if it has not already been loaded.

Types of ClassLoaders

The main built-in class loaders are:

1. Bootstrap ClassLoader

Loads core Java classes.

Examples include classes from the fundamental Java runtime modules.

2. Platform ClassLoader

Loads platform classes and libraries.

It replaced the Extension ClassLoader concept in modern Java.

3. Application ClassLoader

Loads application classes from the application classpath or module path.

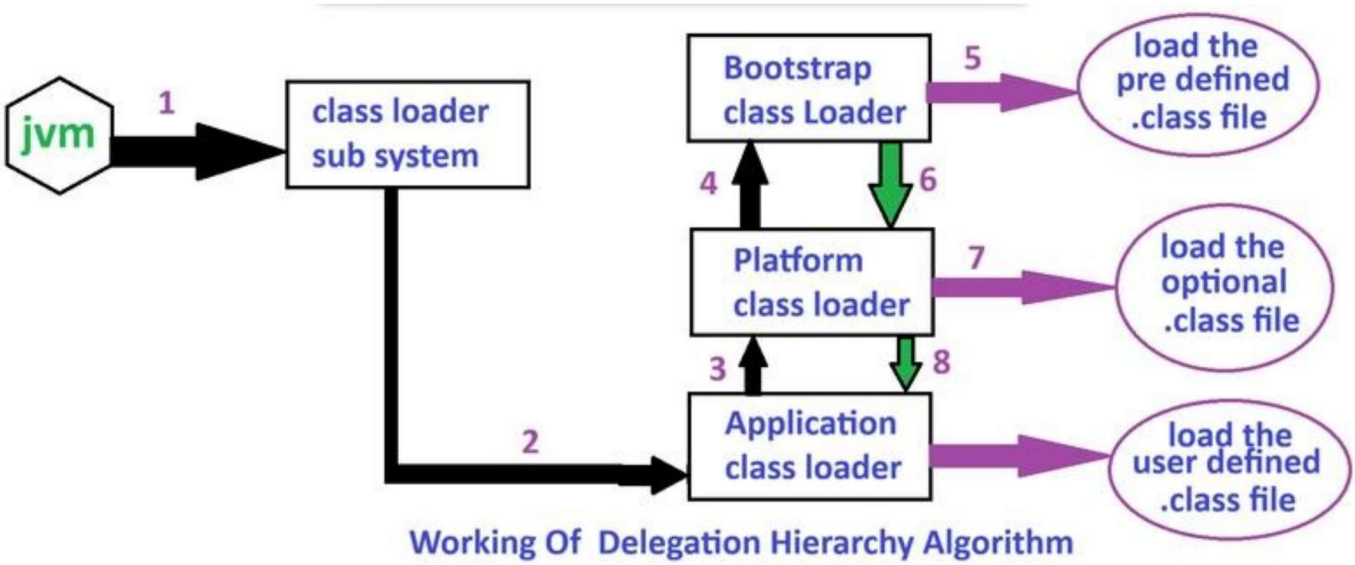


Take your curiosity further

Scan to get the ChatGPT app



ClassLoader hierarchy



What is Parent Delegation?

When a class loader receives a request to load a class, it normally delegates the request to its parent first.

If the parent cannot find the class, the child tries to load it.

This is called parent delegation.

Why is parent delegation important?

1. Avoids loading duplicate core Java classes.
2. Helps maintain security.
3. Prevents application classes from replacing fundamental Java classes in normal loading scenarios.

Take your curiosity further
Scan to get the ChatGPT app

Example

```
public class Main {

    public static void main(String[] args) {

        ClassLoader loader =
                Main.class.getClassLoader();

        System.out.println(loader);

        System.out.println(
                String.class.getClassLoader()
        );
    }
}
```

The second output is:

```
null
```

Because `String` is loaded by the Bootstrap ClassLoader, whose loader reference is represented as null in the standard API.

Real-world usage

Custom class loaders are useful in:

- Application servers.
- Plugin architectures.
- Frameworks.
- Dynamic class loading.
- Isolated application environments.

Take your curiosity further
Scan to get the ChatGPT app

21. What is Garbage Collection in Java?

Garbage Collection (GC) is an automatic memory management process in Java.

It identifies objects that are no longer reachable by the application and reclaims their heap memory.

Why do we need GC?

Suppose:

```
class Employee {
}

class Main {

    public static void main(String[] args) {

        Employee emp = new Employee();

        emp = null;
    }
}
```

Initially:

```
emp → Employee object
```

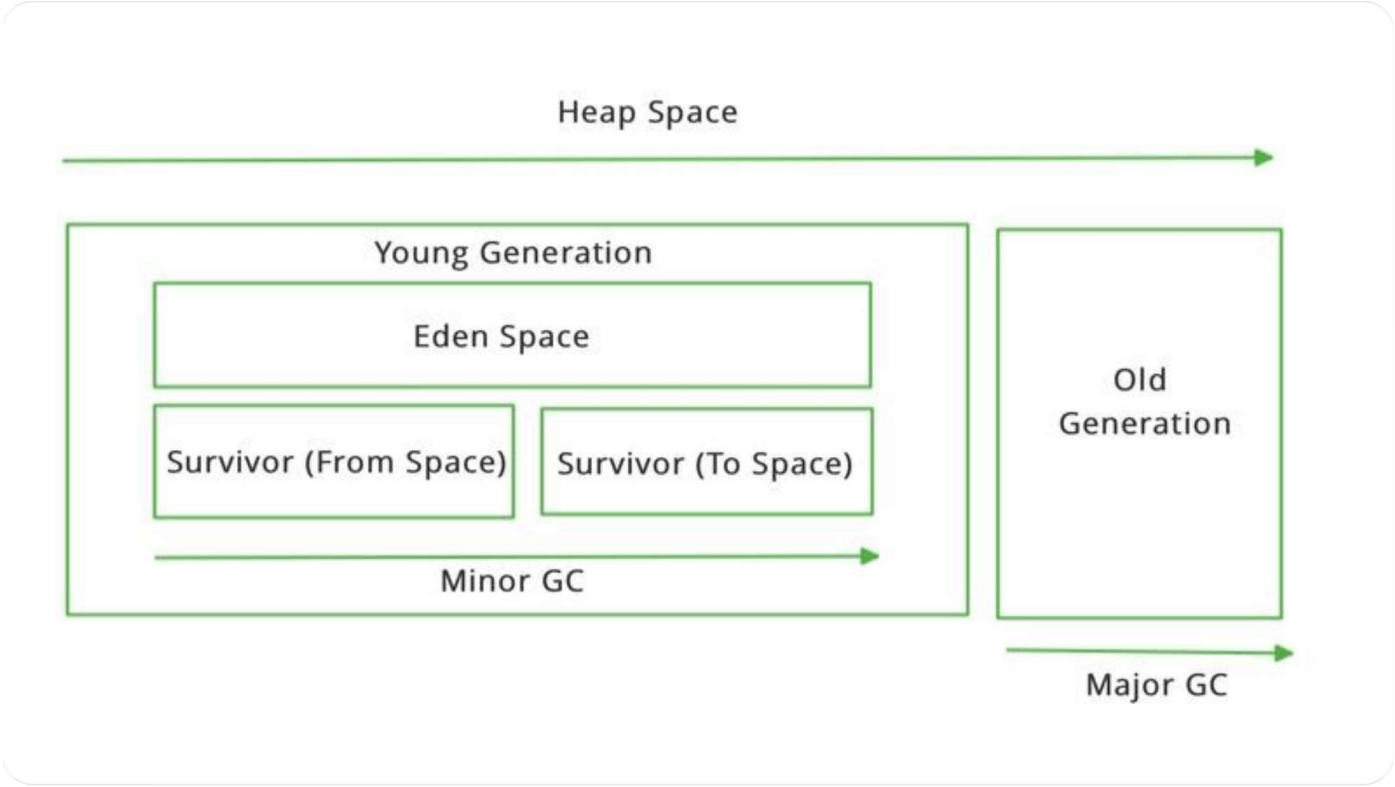
After:

```
emp = null;
```

Take your curiosity further
Scan to get the ChatGPT app

The previous Employee object becomes eligible for garbage collection if there are no other reachable references to it.

How does Garbage Collection work?



Conceptually, many GC implementations organize heap memory into generations.

1. Young Generation

Newly created objects generally start here.

It contains:

- Eden space.
- Survivor spaces.

2. Old Generation

Objects that survive multiple young-generation collections may be promoted here.

Take your curiosity further
Scan to get the ChatGPT app

3. Metaspace

Stores class metadata and is outside the Java heap in modern HotSpot JVMs.

GC process

1. Identify reachable objects from GC roots.
2. Determine which objects are no longer reachable.
3. Reclaim memory.
4. Depending on the collector, move or compact objects to reduce fragmentation.

What are GC roots?

Examples include:

- Active thread stacks.
- Static references.
- JNI references.
- Other JVM-maintained references.

Can we force garbage collection?

No. We can request it:

```
System.gc();
```

But JVM is not required to perform garbage collection immediately.

Can an object be eligible for GC while its variable is still in scope?

Yes.

An object can become unreachable even before the variable's lexical scope ends, depending on whether the reference is still considered live.

Take your curiosity further
Scan to get the ChatGPT app

Important interview point

Java prevents many manual memory-management errors, but memory leaks are still possible.

Example:

```
static List<Object> list = new ArrayList<>();
```

If objects are continuously added and never removed, the static list retains references and prevents those objects from being collected.

Real-world examples of memory leaks:

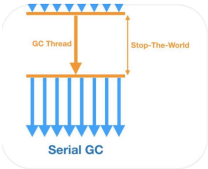
- Unbounded caches.
- Static collections.
- Listeners not unregistered.
- Incorrect ThreadLocal cleanup.
- Long-lived references to large objects.

22. What are Garbage Collectors?

Garbage collectors are JVM components responsible for reclaiming memory from unreachable objects.

Different garbage collectors are designed for different performance requirements.

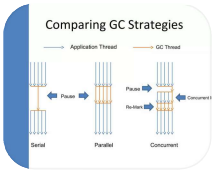
Main Java Garbage Collectors



1. Serial GC

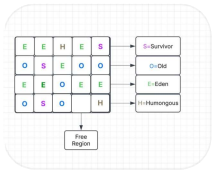
Uses a single thread for garbage collection. Suitable for small applications with limited heap and simple workloads.

Take your curiosity further
Scan to get the ChatGPT app



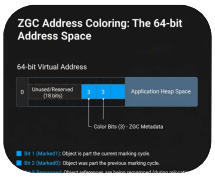
2. Parallel GC

Uses multiple threads for garbage collection. Focuses on throughput and is useful when maximum application throughput is important.



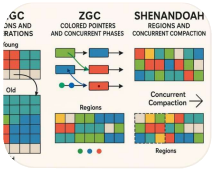
3. G1 GC

Divides the heap into regions and prioritizes regions with more reclaimable garbage. Designed to balance throughput and predictable pause times.



4. ZGC

A low-latency garbage collector designed to keep pauses very short, even with large heaps.



5. Shenandoah GC

Focuses on reducing pause times by performing much of its work concurrently with application threads.

Comparison

Garbage Collector	Main objective
Serial GC	Simple, small workloads
Parallel GC	High throughput
G1 GC	Balanced throughput and pause time
ZGC	Very low latency
Shenandoah	Low pause times

Take your curiosity further
Scan to get the ChatGPT app

How to enable a garbage collector?

Serial GC

```
java -XX:+UseSerialGC Main
```

Parallel GC

```
java -XX:+UseParallelGC Main
```

G1 GC

```
java -XX:+UseG1GC Main
```

ZGC

```
java -XX:+UseZGC Main
```

Shenandoah

```
java -XX:+UseShenandoahGC Main
```

Availability depends on the JVM distribution and version.

How to check which GC is being used?

```
java -XX:+PrintCommandLineFlags -version
```

For GC logs:



Take your curiosity further

Scan to get the ChatGPT app



```
java -Xlog:gc* Main
```

Real-world scenario: Banking application

Imagine your banking application processes millions of transactions.

During processing:

- Temporary DTOs are created.
- Request objects are generated.
- Response objects are created.
- Transaction-related objects become unreachable after processing.

The garbage collector reclaims memory occupied by unreachable objects.

If GC pauses are too high, application response time can increase.

As a senior developer, you should understand:

- Heap utilization.
- GC pause time.
- Allocation rate.
- Object retention.
- Memory leaks.
- Throughput and latency.

Interview-ready answer:

"Garbage collectors are JVM components that automatically reclaim heap memory from unreachable objects. Java provides different collectors such as Serial, Parallel, G1, ZGC, and Shenandoah. We choose a collector based on application requirements, particularly throughput, heap size, and acceptable GC pause times."



Take your curiosity further

Scan to get the ChatGPT app



Important: Additional Senior-Level Interview Questions

The 22 questions above cover your screenshot. But for a 5-year Java developer interview, interviewers frequently ask the following follow-ups.

1. Difference between HashMap and ConcurrentHashMap?

- HashMap is not thread-safe.
- ConcurrentHashMap supports concurrent access.
- HashMap allows null keys and values.
- ConcurrentHashMap does not allow null keys or values.
- ConcurrentHashMap provides better concurrent behavior than synchronizing an entire Hashtable for typical concurrent workloads.

2. Difference between synchronized and Lock?

synchronized	ReentrantLock
Built-in Java keyword	Explicit locking API
Automatically releases monitor when leaving synchronized block	Must explicitly unlock
No timed lock acquisition	Supports tryLock with timeout
Supports intrinsic monitor wait/notify	Supports Condition objects
Simpler syntax	More flexible

Example:

```
Lock lock = new ReentrantLock();

try {

    lock.lock();

    System.out.println("Critical section");
```

Take your curiosity further
Scan to get the ChatGPT app

```
} finally {

    lock.unlock();
}
```

3. Difference between ArrayList and LinkedList?

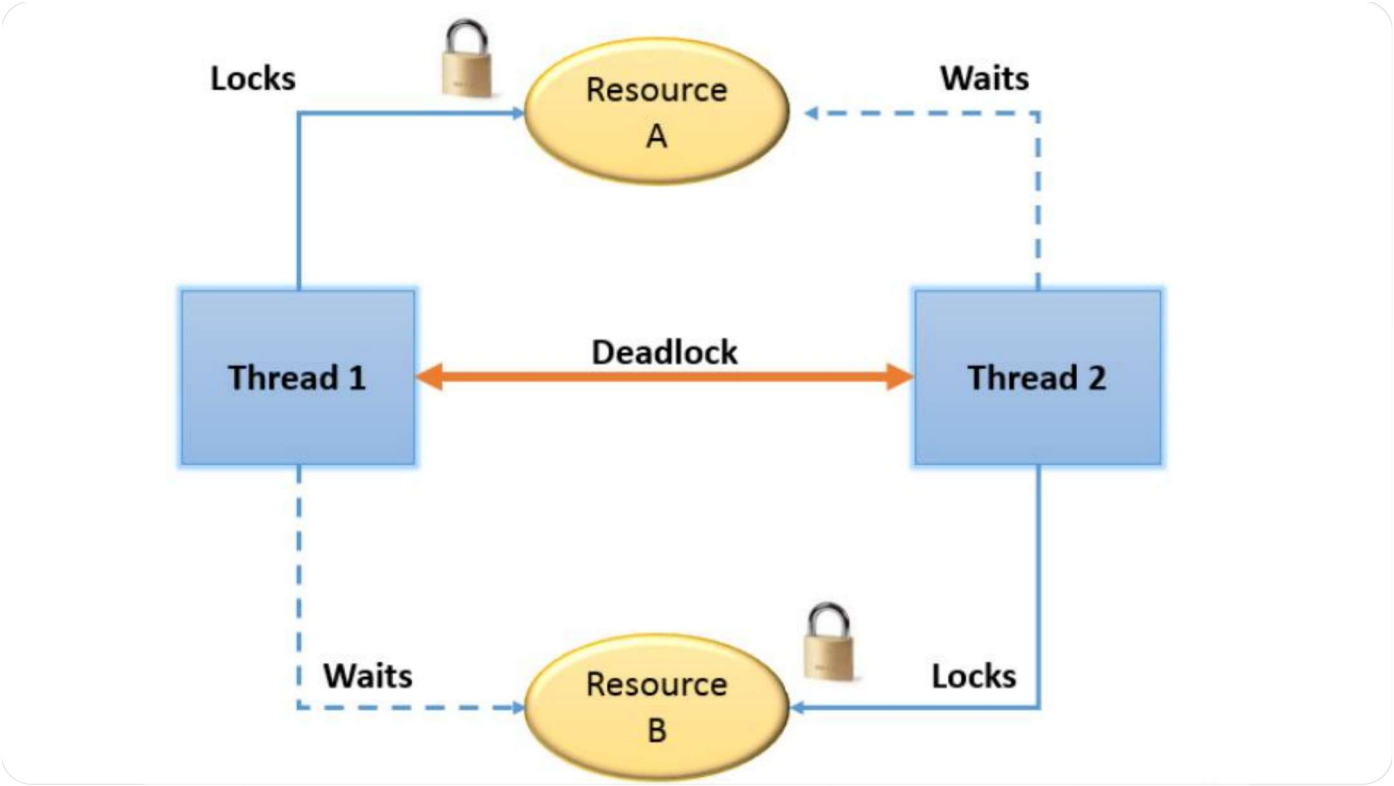
ArrayList	LinkedList
Dynamic array	Doubly linked list
Fast indexed access	Slow indexed access
Insertion in middle can require shifting	Insertion can be efficient once position is reached
Less memory overhead	More memory overhead
Usually preferred for general list operations	Useful for specific deque/list workloads

4. What is a Deadlock?

Deadlock occurs when two or more threads wait indefinitely for resources held by each other.

Example:

Take your curiosity further
Scan to get the ChatGPT app



Thread 1 holds Lock A and waits for Lock B.

Thread 2 holds Lock B and waits for Lock A.

Neither can proceed.

Prevention:

- Acquire locks in a consistent order.
- Avoid unnecessary nested locks.
- Use timed lock acquisition.
- Keep synchronized sections small.

5. What is the difference between Future and CompletableFuture?

Future	CompletableFuture
Represents asynchronous result	Represents asynchronous and composable result

Take your curiosity further
Scan to get the ChatGPT app

Limited chaining	Supports chaining
Usually uses blocking get()	Supports callbacks and composition
Limited exception handling	Rich exception-handling methods
Introduced in Java 5	Introduced in Java 8

Final Interview Revision Sheet

Use this table to revise all 22 questions before an interview.

No.	Topic	Most important point
1	Constructor	Initializes objects
2	final/finally/finalize	Keyword vs block vs deprecated method
3	Overloading/overriding	Compile-time vs runtime polymorphism
4	Static overriding	Method hiding
5	Final constructor	Not allowed
6	Main overloading	Yes, JVM uses standard signature
7	Interface object	Through implementation, anonymous class or lambda
8	Final variable	Reference cannot be reassigned
9	Java 8 features	Lambda, streams, Optional
10	CompletableFuture	Asynchronous task composition
11	Lambda	Anonymous function
12	Functional interface	One abstract method
13	Built-in interfaces	Predicate, Function, Consumer, Supplier

Take your curiosity further
Scan to get the ChatGPT app

14	HashMap	Key-value data structure
15	HashMap internal	Hashing, buckets, collisions, resizing
16	Multithreading	Concurrent execution
17	Achieving multithreading	Thread, Runnable, ExecutorService
18	Thread creation	Thread, Runnable, Callable
19	JRE architecture	Runtime environment and JVM
20	ClassLoader	Loads classes
21	Garbage Collection	Reclaims unreachable objects
22	Garbage Collectors	Serial, Parallel, G1, ZGC, Shenandoah

Test Your Interview Preparation

Since these are interview questions, try answering them yourself. I can evaluate your answers as an interviewer for a 5-year experienced Java Developer position.

Java interview mock test 1/8 answered

Q1. What happens internally when you insert a key into HashMap?

- HashMap directly stores the key in an array
- HashMap calculates hash, finds a bucket and handles collisions
- HashMap sorts all keys
- HashMap creates a new thread

Q2. Can we override a static method in Java?

Take your curiosity further
Scan to get the ChatGPT app

- Yes, always
- No, static methods are hidden, not overridden
- Only with final keyword
- Only with private methods

Q3. Which functional interface accepts an argument and returns boolean?

- Function
- Consumer
- Predicate
- Supplier

Q4. Which method starts a new Java thread?

- run()
- start()
- execute()
- call()

Q5. What happens when a HashMap key is modified after insertion?

- HashMap automatically updates its bucket
- The map always removes the key
- Lookup may fail because the hash or bucket can change
- The JVM prevents all modifications

Take your curiosity further
Scan to get the ChatGPT app

Q6. Which CompletableFuture method transforms a result?

- thenApply()
- thenRun()
- runAsync()
- join()

Q7. Which memory area stores Java objects?

- Java Stack
- Heap
- PC Register
- Native Method Stack

Q8. What is the default HashMap load factor?

- 0.25
- 0.50
- 0.75
- 1.5

Submit answers

My recommendation: For a 5-year Java developer interview, give special attention to HashMap internal working, Java 8 streams, CompletableFuture, ExecutorService, synchronization, JVM memory architecture, and garbage collection. Interviewers often go deeper into these topics by asking you to write code and explain real project scenarios.

Take your curiosity further
Scan to get the ChatGPT app

If you want, I can:

- Explain next core Java question
- Give interview tips for CompletableFuture
- Explain more functional interfaces with examples

Take your curiosity further
Scan to get the ChatGPT app

Take your curiosity further
Scan to get the ChatGPT app